Learning Objectives

This lab lets you practice looping over dictionaries and problem-solving using nested dictionaries.

Estimated Size

1 program file: `weather_stations.py`, 2 functions.

Setup

In your Python files, you must include author information. On the first line, add a `# Author` comment line, where you mention your full name. Similarly, include your email and Spire ID on the `# Email` and `# Spire ID` comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author   : John Snow
# Email    : jsnow@umass.edu
# Spire ID : 12345678
```

This lab has a starter code named `weather_stations.py`. You can download it here. This is the only file you need to work on and submit. **Modify the comments section at the beginning of the file to correctly include all your team members' information**.

0. Weather Stations

Weather service providers such as Weather Underground usually employ a large number of weather stations all over the world to collect weather data each day. Each station has a unique name or ID, a GPS coordinate (its location), and additional information such as the sponsor/owner of the station. For example, UMass CICS owns a couple of weather stations located on top of the CS building.

In this lab you will use loops and dictionaries to implement several common tasks that weather service providers routinely perform. The starter code contains two **global dictionaries**. The first is called `stations`, where the key is each station's unique ID, and the value is itself a dictionary storing the GPS location (required) of the station and additional data like the sponsor (optional). For example:

```
stations = {'UMCS1': {'gps': (42.3949101,-72.5304337), 'sponsor':'UMass CICS'},
            'BOSTA': {'gps': (42.2140122,-71.3254988), 'sponsor':'UMass Boston'},
            'AMHER': {'gps': (42.3951000,-72.5320000)} }
```

A GPS coord is a 2-element tuple in the form of `(latitude, longitude)`. For each station, the GPS coord is guaranteed to exist while the sponsor may or may not exist.

The second dictionary is called `data_today`. It is also a nested dictionary where the key is a station's unique ID, and the value is itself a dictionary storing recorded weather data today, such as temperature (in Celsius or Fahrenheit), humidity, wind, precipitation, solar radiation etc. For example:

```
data_today = {'BOSTA': {'tempC': 12.2, 'humidity': 59},
              'UMCS3': {'humidity': 33, 'wind': 25},
```

```
            'AMHER': {'tempF': 64.4} }
```

When dealing with real-world data, you have to get used to the fact that the data may be incomplete and inconsistent. For example, not all stations report temperatures; of those that report, some report in Celsius (`tempC`) while others report in Fahrenheit (`tempF`). In addition, some stations may be offline/broken today, therefore not reporting any data at all (hence they do not have an entry in `data_today`).

1. Implement function `get_stations_by_loc` (8 points)

Write a function called `get_stations_by_loc`. It takes two parameters: a **target** (query) location, with a default value *(0, 0)*, and a **radius** (in meters), with a default value of *1000* (1km); and it **returns a set of stations** whose distance is within (`<=`) the radius to the target (query) location. The starter code provides a function called `distance()` that allows you to calculate the distance (in meters) between two GPS coordinates. This function basically computes an arc length on Earth's sphere, and you do NOT need to modify this function. Specifically, your function should:

1. Take a `target` GPS location (a 2-element tuple) and a `radius` as parameters.
2. Loop over the `stations` dictionary, for each station calculate its distance to the target by using the station's GPS coordinates, and calling the `distance()` function; then compare the result with the `radius`. All stations within radius of the target are collected into a **set**. At the end of the loop, return the **set**.
3. If there are no stations within the radius of the target, the function naturally returns an empty set.
4. Do **NOT** ask the user for any input. Do **NOT** print anything in your function.
5. If you see **missing required positional arguments** on Gradescope, you may not be implementing the default parameters correctly.

Note that an empty set is created using `set()`. In contrast, `{}` creates an empty dictionary. Note `stations` is a global object, so you can (and should) use it in this function. Below are example outputs:

```
print(get_stations_by_loc((42.39,-72.53), 1))      # search radius 1 m
Output → set()                                      # empty set
```

```
print(get_stations_by_loc((42.39,-72.53))   # search radius 1 km by default
Output → {'AMHER', 'UMCS2', 'UMEN1', 'UMCS1'}
```

```
print(get_stations_by_loc(radius=200000, target=(42.39,-72.53)) # search radius 200 km
Output → {'BOSTB', 'AMHER', 'UMCS2', 'UMEN1', 'UMCS1', 'BOSTA', 'H2B4A', 'BOSTH'}
```

```
print(get_stations_by_loc(radius=10**6)) # search radius 1000 km from default target(0, 0)
Output → set()                           # empty set
```

Because a set is unordered, your output may look different, but it should contain the same set of stations as the above. ***Tip***: If you choose to use set comprehension, you can write this function in 1 line.

2. Implement function `get_avg_temp` (7 points)

Write a function called `get_avg_temp` which takes a **target** (query) location, with a default value *(0, 0)*, a **radius,** with a default value *1000* (1km), a boolean parameter **celsius**, with a default value *True*, and returns the **average temperature (in Celsius** or **Fahrenheit)** calculated using weather stations within the

radius to the target. This is a basic task of any weather service provider: as we don't have a weather station at every query point, to calculate the temperature, we need to find available weather stations within a certain radius of the query point, and use them to calculate the average temperature. To do so, your function should:

1. Take a `target` GPS location (a 2-element tuple), a `radius`, and a `celsius` parameter.
2. Call `get_stations_by_loc` you implemented above, to obtain a set of stations nearby; then loop over this set, for each station in this set, look up the `data_today` dictionary to get its reported temperature value today.
3. If parameter **celsius** is True, calculate the **average** temperature in **Celsius** and return it. Otherwise, calculate the average temperature in **Fahrenheit** and return it.
4. Note that not all stations report data today; if they do, they may not report temperatures. Also, stations may report temperature either in Fahrenheit (`tempF`) or Celsius(`tempC`), in which case you need to convert that to the corresponding format (the formula is given below). Your code needs to handle all these cases. In the case you cannot compute temperature (either because Step 2 returned an empty set, or none of the stations returned in Step 2 exists in `data_today`, or none reports temperature), this function should return `None` to indicate it's unable to fulfill the request.
5. Do **NOT** ask the user for any input. Do **NOT** print anything in your function.
6. If you see **missing required positional arguments** on Gradescope, you may not have implemented the default parameter correctly.

You can assume a station will NOT report both `tempC` and `tempF` at the same time. Note that `data_today` is a global object, so you can (and should) use it in this function. Below are example outputs:

```
print(get_avg_temp((42.39,-72.53), 1))
Output → None      # search radius too small, resulting in no stations found
```

```
print(get_avg_temp((42.39,-72.53)))
Output → 13.0      # search radius 1 km and return the temperature in Celsius by default
```

```
print(get_avg_temp(radius=50000, target=(42.214,-71.325))
Output → 11.85     # search radius 50 km and return the temperature in Celsius by default
```

```
print(get_avg_temp((42.214,-71.325), 50000, celsius=False))
Output → 53.33     # search radius 50 km and return the temperature in Fahrenheit
```

```
print(get_avg_temp(radius=10**6))
Output → None                    # none of the stations found reports temperature
```

Tip: Use the `in` operator to check if a key exists in a dictionary. For example, `name in data_today` tells you whether a station `name` exists in `data_today`. Similarly, you can use `in` to check if the key `'tempC'` or `'tempF'` exists in the station's data.

The formula to convert `tempF` to `tempC` is: `tempC = (tempF - 32) * 5 / 9`

The formula to convert `tempC` to `tempF` is: `tempF = tempC * 9 / 5 + 32`

If you notice any small floating-point errors (such as `13.000001` instead of `13.0`), you can ignore that. The autograder will account for such small errors when comparing your result with the reference.

3. Submission to Gradescope

Log in to Gradescope, select **Lab 09: advanced function with dictionaries (Code)**, and submit the required file. Wait until the autograder finishes running and returns the result to you. You can resubmit as many times as you like before the deadline.

Also, remember to submit the lab questionnaire. If you see a question you do not know how to answer, ask TAs/UCAs in your lab, use Piazza, or go to one of our office hours.

(Common Paragraph) Academic Honesty

All work that is completed in this assignment is your own (if you work individually) or your own group's (if you work in a group). You may talk to other students about the problems, and brainstorm about solutions. However, you may NOT share code in any way, except with your group members. What you submit **must be your own or your own group's work**.

You may not use any code that is posted on the internet. If you are not sure it is in your best interest to contact the course staff. We will be using software that will compare your code to other students in the course as well as online resources. It is very easy for us to detect similar submissions and will result in a failure for the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the UMass Academic Honesty Policy and Procedures so you are aware of what this means.

Copying solutions, whether partial or whole, obtained from other students or elsewhere, is academic dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty you should re-read the course policies. We assume you have read the course policies in detail and by submitting this project you have provided your virtual signature in agreement with these policies.